

José Eduardo Gutiérrez Conejo - 2019073558

Bases de Datos II

Lectura 8/9: Consistency Models of NoSQL Databases

1. Explique la diferencia entre modelos de consistencia data-centric y client-centric

Data-centric: Este modelo se enfoca en cómo los datos son almacenados y replicados en un sistema distribuido. Se asegura de que los datos sean consistentes entre múltiples nodos o servidores.

Client-centric: Este modelo se centra en cómo un cliente percibe la consistencia de los datos. Se enfoca en cómo un cliente interactúa con el sistema y qué tipo de garantías de consistencia puede esperar.

2. Comente similitudes y diferencias entre los modelos de consistencia de Redis y Cassandra

Redis: Este tiende a priorizar la velocidad y la baja latencia en sus operaciones. Utiliza un modelo de consistencia más relajado, lo que significa que puede haber cierto retraso en la propagación de los cambios a través de los nodos. Esto hace que Redis sea muy eficiente para aplicaciones que necesitan respuestas rápidas.

Cassandra: Este se centra en la disponibilidad y la escalabilidad en entornos distribuidos. Utiliza un modelo de consistencia más estricto, lo que significa que garantiza que los datos estén siempre actualizados y disponibles en todos los nodos, aunque esto pueda llevar a una pequeña penalización en términos de latencia.

3. Comente como afecta el rendimiento y funcionamiento de una base de datos los siguientes modelos de consistencia:

Strong Consistency: Esto puede llevar a una mayor latencia. Ofrece la garantía más alta de consistencia, lo que significa que los datos siempre estarán actualizados. Sin embargo, puede limitar la escalabilidad y la tolerancia a fallos. Tiene un impacto en el rendimiento ya que cada operación de lectura espera a que todas las réplicas estén actualizadas antes de devolver un resultado.

Weak Consistency: Esto puede resultar en menor latencia. Permite que las lecturas muestren datos más antiguos y puede haber un retraso en la propagación de las escrituras. Ofrece una mayor escalabilidad, pero a costa de la consistencia inmediata. Tiene un impacto positivo en el rendimiento ya que las operaciones no esperan la actualización de todas las réplicas antes de completarse.

Eventual Consistency: Garantiza que, después de un tiempo suficientemente largo, todas las réplicas convergerán en el mismo estado. Puede haber un período donde las lecturas muestren versiones desactualizadas. Tiene un impacto positivo en el rendimiento, ya que no requiere que todas las réplicas estén actualizadas antes de completar una operación. Esto puede resultar en una alta escalabilidad y baja latencia.

Causal Consistency: Asegura que las operaciones causales sean vistas en el mismo orden por todos los nodos, proporcionando una noción de causalidad entre las operaciones. Puede combinar eficiencia y consistencia en ciertos casos de uso. Tiene un impacto en el rendimiento, pero es menos restrictivo que Strong Consistency.

Read-your-writes Consistency: Asegura que un cliente siempre vea sus propias escrituras inmediatamente después de realizarlas. Tiene un impacto en el rendimiento ya que garantiza que un cliente siempre vea sus propias escrituras. Puede haber una pequeña latencia adicional.

Session Consistency: Ofrece una garantía específica para las interacciones de un usuario dentro de una sesión. Puede tener un impacto en el rendimiento ya que garantiza que todas las operaciones dentro de una sesión de cliente se vean en el mismo orden. Puede haber una latencia adicional.

Monotonic Reads Consistency: Puede haber una pequeña latencia adicional. Garantiza una progresión lógica en las lecturas de un cliente. Puede tener un impacto en el rendimiento ya que asegura que un cliente siempre vea los datos en un orden creciente.

Monotonic Writes Consistency: Esencial para garantizar que las escrituras no se pierdan o sobrescriban de manera inesperada. Puede tener un impacto en el rendimiento ya que asegura que las escrituras se hagan en un orden creciente. Puede haber una pequeña latencia adicional.